

LAB EXERCISE 2

Design and Implement a Basic URL Shortener Using Spring Boot

AIM

To design and implement a basic URL Shortener application using Spring Boot and MongoDB that generates a short URL for a given long URL using a hashing technique, and redirects users to the original URL.

Team Details

S.No	Name	Register Number
1	Dinesh Kumar J	3122245001042
2	Gopinath S	3122245001048
3	Harish R	3122245001055
4	Srikrishna O S	3122245001312

PROCEDURE

The URL shortener was implemented by following the steps described in the subsections below.

1. System Architecture

The application follows a simple client-server architecture built directly on Spring Boot's MVC layer. A client (browser or API client) sends an HTTP request to the server. UrlShortenerController exposes both REST endpoints and contains the hashing and uniqueness-checking logic directly. The controller talks to UrlMappingRepository, a Spring Data MongoDB interface, which persists and retrieves URL mappings from a local MongoDB database. Client (Browser / API Client) -> UrlShortenerController (REST endpoints + SHA-256 hashing + uniqueness check) -> UrlMappingRepository (Spring Data MongoDB) -> MongoDB

2. Database Collection Design

Field	Type	Description
id	ObjectId (auto-generated)	Unique document identifier
shortCode	String	Generated short code (6-8 URL-safe Base64 characters)
longUrl	String	The original long URL

3. REST API Design

Method	Endpoint	Description
POST	/shorten	Accepts JSON { "longUrl": "..." } and returns HTTP 201 with JSON { "shortUrl": "..." }
GET	/ {shortCode}	Looks up the short code and issues an HTTP 302 redirect to the original long URL. Returns 404 if not found.

4. How the Short Code is Generated

The long URL is hashed using SHA-256. The 32-byte digest is then encoded with URL-safe Base64 (`java.util.Base64.getUrlEncoder()`, no padding), and the first 6 characters of that encoding are used as the short code. Before saving, the code checks the MongoDB collection for a collision: if the 6-character code is already taken, it tries 7, then 8 characters of the same Base64 encoding.

5. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>com.ssn</groupId>
  <artifactId>url-shortener</artifactId>
  <version>1.0.0</version>
  <name>url-shortener</name>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-mongodb</artifactId>
```

```
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <source>17</source>
                    <target>17</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

6. application.properties

```
server.port=8080

# MongoDB - local instance running on the default port
spring.data.mongodb.uri=mongodb://localhost:27017/urlshortener
```

7. UrlMapping.java (MongoDB Document)

```
package com.ssn.urlshortener;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "url_mapping")
public class UrlMapping {

    @Id
    private String id;

    private String shortCode;
    private String longUrl;

    public UrlMapping() {
```

```
}

public UrlMapping(String shortCode, String longUrl) {
    this.shortCode = shortCode;
    this.longUrl = longUrl;
}

public String getId() {
    return id;
}

public String getShortCode() {
    return shortCode;
}

public void setShortCode(String shortCode) {
    this.shortCode = shortCode;
}

public String getLongUrl() {
    return longUrl;
}

public void setLongUrl(String longUrl) {
    this.longUrl = longUrl;
}
}
```

8. UrlMappingRepository.java

```
package com.ssn.urlshortener;

import org.springframework.data.mongodb.repository.MongoRepository;
import java.util.Optional;

public interface UrlMappingRepository extends MongoRepository<UrlMapping, String> {

    Optional<UrlMapping> findByShortCode(String shortCode);
}
```

9. UrlShortenerController.java

```
package com.ssn.urlshortener;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Base64;
import java.util.Optional;

@RestController
public class UrlShortenerController {

    private final UrlMappingRepository repository;

    public UrlShortenerController(UrlMappingRepository repository) {
        this.repository = repository;
    }

    @PostMapping("/shorten")
    public ResponseEntity<ShortenResponse> shorten(@RequestBody ShortenRequest request) {

        String longUrl = request.getLongUrl();
        String shortCode = generateUniqueShortCode(longUrl);

        UrlMapping mapping = new UrlMapping(shortCode, longUrl);
        repository.save(mapping);

        String shortUrl = "http:
ShortenResponse response = new ShortenResponse(shortUrl);

        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/{shortCode}")
    public ResponseEntity<Void> redirect(@PathVariable String shortCode) {

        Optional<UrlMapping> result = repository.findByShortCode(shortCode);

        if (result.isEmpty()) {
            return ResponseEntity.notFound().build();
        }
    }
}
```

```
String longUrl = result.get().getLongUrl();

return ResponseEntity.status(HttpStatus.FOUND)
    .header(HttpHeaders.LOCATION, longUrl)
    .build();
}

private String generateUniqueShortCode(String longUrl) {

    String hash = sha256Base64(longUrl);
    int length = 6;
    String code = hash.substring(0, length);

    while (repository.findByShortCode(code).isPresent() && length < 8) {
        length++;
        code = hash.substring(0, length);
    }

    return code;
}

private String sha256Base64(String input) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));

        return Base64.getUrlEncoder().withoutPadding().encodeToString(hashBytes);

    } catch (Exception e) {
        throw new RuntimeException("Error generating hash", e);
    }
}
}
```

SCREENSHOTS

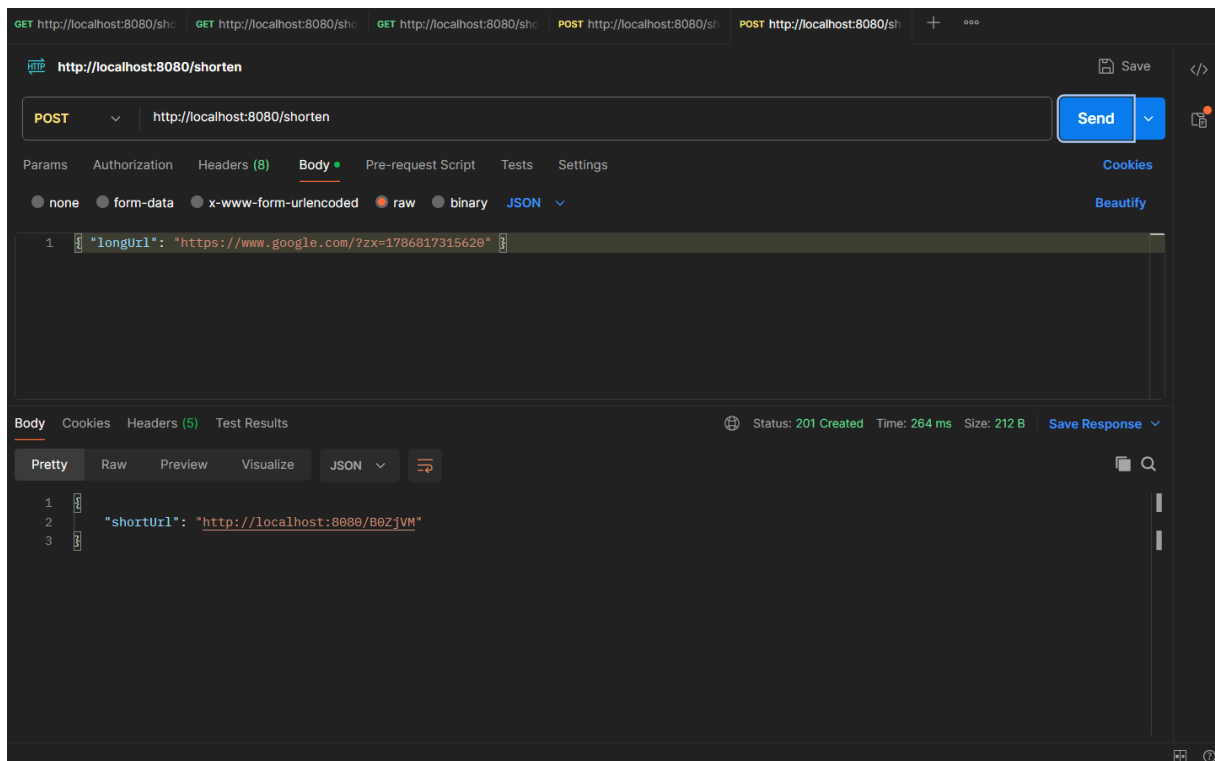


Fig 1 - POST /shorten request and response in the API client

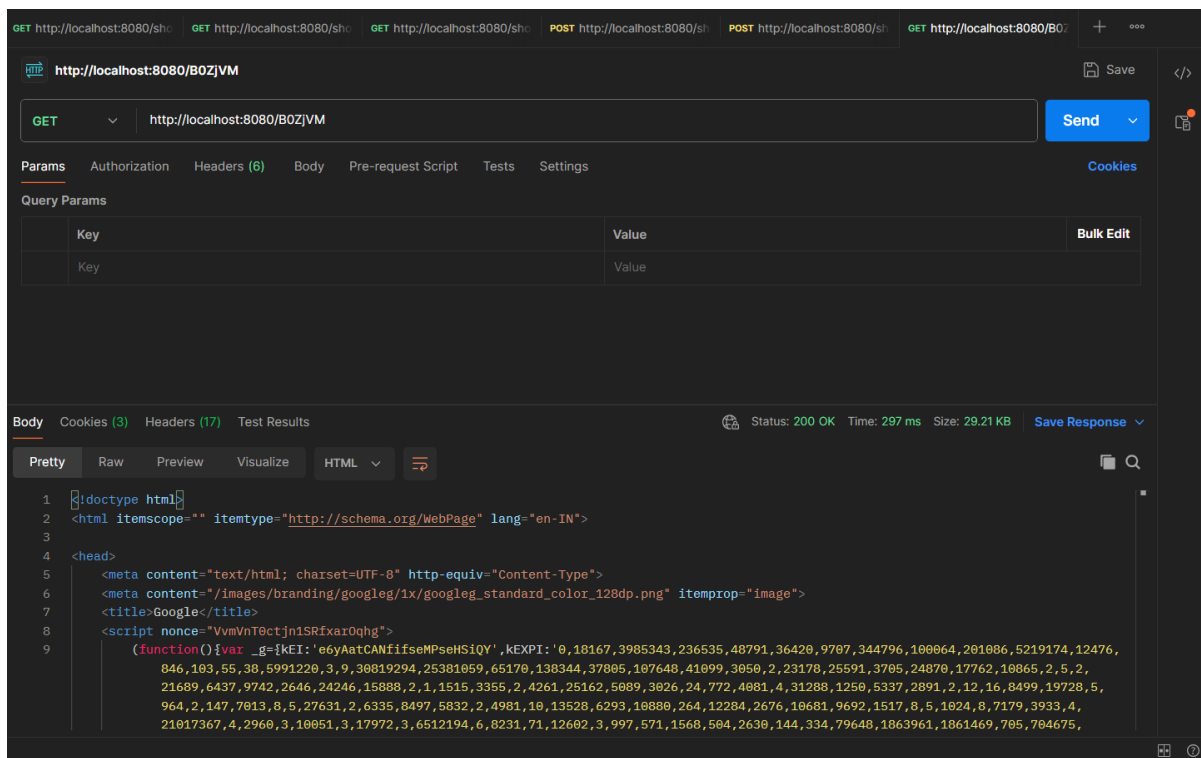


Fig 2 - Short URL opened in a web browser; it followed the 302 redirect to the original long URL

TEST CASES

The application was built and run locally (mvn spring-boot:run, port 8080) using a local MongoDB instance. It was tested with real HTTP requests using both an API client and a web browser. A PowerShell benchmark script ran 10 POST /shorten requests and 10 GET

/ {shortCode} requests against the running application.

```
POST /shorten
curl -i -X POST http://localhost:8080/shorten -H "Content-Type: application/json" -d '{"longUrl": "https://www.google.com/search?q=system+design"}'

HTTP/1.1 201 Created
Content-Type: application/json
{"shortUrl": "http://localhost:8080/Ayirwc"}

GET / {shortCode}
curl -i http://localhost:8080/Ayirwc

HTTP/1.1 302 Found
Location: https://www.google.com/search?q=system+design
Content-Length: 0
```

Endpoint	Runs	Average (ms)	Success
POST /shorten	10	11.74	10/10
GET / {shortCode}	10	2.91	10/10

LEARNING OUTCOMES

- Understand the basic workflow of a URL Shortener.
- Design a simple client-server architecture using Spring Boot.
- Develop REST APIs using Spring Boot.
- Use Spring Data MongoDB to store and retrieve data.
- Generate unique short codes using a hashing technique.
- Implement URL redirection using HTTP status codes.
- Test REST APIs using an API client and verify redirection in a web browser.